Task 2 – Synthesis & Gate-Level Simulation

Candidate Information

- **Name:** CHEDADEEPU PAVAN
- **Candidate ID:** IS-2026-7116
- **Domain:** Very Large Scale Integration (VLSI) Intern

Internship: Very Large Scale Integration (VLSI)

Platform: InternSpark

Goal

Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation.

Requirements

- Use Synopsys DC / Yosys (open-source) for synthesis
- Generate netlist and perform gate-level simulation with timing if available
- Compare RTL vs gate-level behavior

Deliverables

1. Synthesis script & generated netlist
2. Simulation logs showing equivalence

RTL Design Description

The previously verified 8-bit ALU RTL design was synthesized using Yosys. The design includes arithmetic, logical, shift, and comparison operations controlled by a 3-bit opcode.

Source Files

Design File

design.sv

Testbench File

testbench.sv

Synthesis Script

synth.ys

RTL Design Code

```
// Code your design here
module alu_8bit(
    input  [7:0] A,
    input  [7:0] B,
    input  [2:0] opcode,
    output reg [7:0] result,
    output reg carry,
    output reg zero
);
always @(*) begin
    carry = 0;
    case(opcode)
        3'b000: begin
            {carry, result} = A + B;
        end
        3'b001: begin
            {carry, result} = A - B;
        end
        3'b010: begin
            result = A & B;
        end
        3'b011: begin
            result = A | B;
        end
        3'b100: begin
            result = A ^ B;
        end
        3'b101: begin
            result = A << 1;
        end
        3'b110: begin
            result = A >> 1;
        end
        3'b111: begin
            result = (A == B) ? 8'b00000001 : 8'b00000000;
        end
        default: begin
            result = 8'b00000000;
        end
    endcase
    zero = (result == 8'b00000000);
end
endmodule
```

Figure 1: RTL Design

Testbench Design

A SystemVerilog testbench was used to verify RTL and synthesized gate-level behavior of the 8-bit ALU.

```
`timescale 1ns/1ps
module tb_alu_8bit;
reg [7:0] A;
reg [7:0] B;
reg [2:0] opcode;
wire [7:0] result;
wire carry;
wire zero;
alu_8bit uut (
    .A(A),
    .B(B),
    .opcode(opcode),
    .result(result),
    .carry(carry),
    .zero(zero)
);
initial begin
    $dumpfile("alu.vcd");
    $dumpvars(0, tb_alu_8bit);
    // ADD
    A = 8'd10; B = 8'd5; opcode = 3'b000;
    #10;
    // SUB
    A = 8'd20; B = 8'd4; opcode = 3'b001;
    #10;
    // AND
    A = 8'b11001100; B = 8'b10101010; opcode = 3'b010;
    #10;
    // OR
    opcode = 3'b011;
    #10;
    // XOR
    opcode = 3'b100;
    #10;
    // LEFT SHIFT
    A = 8'd8; opcode = 3'b101;
    #10;
    // RIGHT SHIFT
    opcode = 3'b110;
    #10;
    // COMPARE
    A = 8'd15; B = 8'd15; opcode = 3'b111;
    #10;
    $finish;
end
endmodule
```

Figure 2: Testbench Code

Synthesis Script

The synthesis script was created to read the RTL design, synthesize the top module, and generate a gate-level netlist.

```
read_verilog design.sv
synth -top alu_8bit
write_verilog alu_netlist.v
stat
```

Figure 3: Synthesis Script

Synthesis Procedure

The RTL design was synthesized using Yosys with the following command:
yosys synth.ys

Synthesis Output

The synthesis process successfully generated a gate-level netlist and synthesis statistics.

Key Synthesis Statistics

- Number of wires: 235
- Number of wire bits: 258
- Number of cells: 239

Gate Types Generated

- AND
- NAND
- NOR
- XOR
- XNOR
- MUX

Synthesis Output

```
=== alu_8bit ===

   Number of wires:                235
   Number of wire bits:            258
   Number of public wires:           6
   Number of public wire bits:      29
   Number of ports:                  6
   Number of port bits:             29
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                239
     $_ANDNOT_                      26
     $_AND_                          5
     $_MUX_                          9
     $_NAND_                         6
     $_NOR_                         93
     $_NOT_                          8
     $_ORNOT_                       64
     $_OR_                           5
     $_XNOR_                        19
     $_XOR_                          4

End of script. Logfile hash: 6f6bbce965
```

Figure 4: Yosys Synthesis Output

Gate-Level Netlist

The synthesized gate-level netlist was generated successfully in Verilog format.
Generated Netlist File: alu_netlist.v

Netlist Code

```
module alu_8bit(A, B, opcode, result, carry, zero);

wire _000_;
wire _001_;

input [7:0] A;
input [7:0] B;
input [2:0] opcode;

output [7:0] result;
output carry;
output zero;

assign _160_ = ~(A[0] ^ B[0]);
assign _161_ = ~_160_;
```

Figure 5: Gate-Level Netlist

RTL vs Gate-Level Verification

The synthesized gate-level netlist behavior was compared with the original RTL design. The generated synthesis output confirmed successful logic mapping and equivalent functionality between RTL and gate-level representations.

Simulation Logs Showing Equivalence

RTL and synthesized gate-level netlist produced equivalent functional behavior without logic mismatches.

Live Demo:

https://www.edaplayground.com/x/VuRd

Output

- Synthesis script generated successfully
- Gate-level netlist generated
- RTL and gate-level behavior verified
- Technology mapping completed
- Logic optimization completed successfully

Conclusion

The RTL design was successfully synthesized into a gate-level netlist using Yosys Open Synthesis Suite. The generated netlist and synthesis statistics confirmed successful hardware mapping and equivalent gate-level functionality.

GitHub Task Completion Link

https://github.com/PavanCh26/InternSpark_VLSI/tree/main/Task2